





## Parameter efficient fine-tuning :-

$$f_i(x) = f_{\theta_i}(x) \odot \underbrace{f_{\phi_i}(x)}$$

↓  
added to a pre-trained model to adapt it

- we generally add two adapters in a transformer layer
  - each adapter has a down-projection and up-projection layer

$$W^D \in \mathbb{R}^{n \times d}$$

$$W^U \in \mathbb{R}^{d \times k}$$

$$f_{\phi}(x) = W^U (\sigma(W^D x))$$

$d \rightarrow$  dimension of model

for  $L$  layers,

$$2L \times (2 \times d_{\text{model}} \times r + d_{\text{model}} + r)$$

# Adapters in parallel :-

→ having a parallel adapter with self-attention  
 $TSch^l = v^0_g(CV^E h^l) \rightarrow$  task specific adapter  
 $h^{l+1} = LN(h^l + SACH^l + TSCh^l)$

## How does it work?

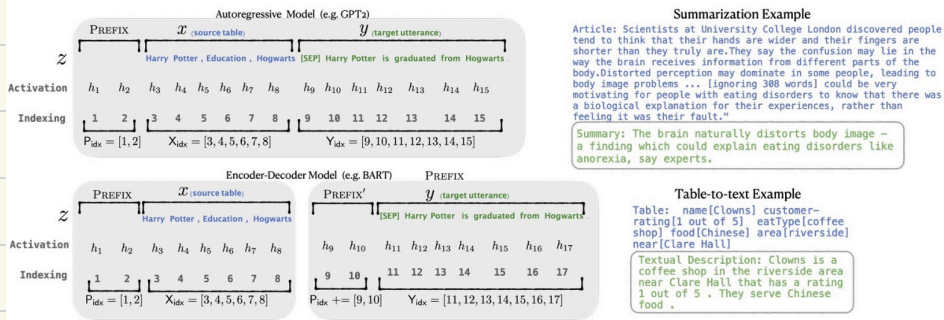


Figure 2: An annotated example of prefix-tuning using an autoregressive LM (top) and an encoder-decoder model (bottom). The prefix activations  $\forall i \in P_{idx}$ ,  $h_i$  are drawn from a trainable matrix  $P_\theta$ . The remaining activations are computed by the Transformer.

<https://aclanthology.org/2021.acl-long.353.pdf>

if only at embedding layer  $\rightarrow d_{model} \times |P_{idx}|$   
 if at all layers  $\rightarrow d_{model} \times |P_{idx}| \times L$

## LORA :-

$W_o + \Delta W = W_o + BA \quad B \in \mathbb{R}^{d_{model} \times d_r}$   $A \in \mathbb{R}^{d_r \times d_{model}}$   
 $frozen \leftarrow W_o \in \mathbb{R}^{d_{model} \times d_{model}}$

## 2-bit quantization :- QLoRA

### QLoRA: Quantization Example for 2-bit

Given a tensor  $X$  of any real data type. We can apply quantization as follows:

1. Normalize  $X$  into the range  $[-1.0, 1.0]$  by dividing by  $\text{absmax}(X)$
2. Find the closest value in the data type (rounding for integers; in general binary search)

Map: {Index: 0, 1, 2, 3 -> Values: -1.0, 0.3, 0.5, 1.0}

*We saw how to get these values for NF4*

Input tensor: [10, -3, 5, 4]

1. Normalize with absmax: [10, -3, 5, 4] -> [1, -0.3, 0.5, 0.4]
2. Find closest value: [1, -0.3, 0.5, 0.4] -> [1.0, 0.3, 0.5, 0.5]
3. Find the associated index: [1.0, 0.3, 0.5, 0.5] -> [3, 1, 2, 2] -> store
4. Dequantization: load -> [3, 1, 2, 2] -> lookup -> [1.0, 0.3, 0.5, 0.5] -> denormalize -> [10, 3, 5, 5]

<https://neurips.cc/media/neurips-2023/Slides/73855.pdf>

## Gradient check-pointing :-

checkpoints are usually placed at every  $\sqrt{n}$  layer, considering we have a  $n$ -layer neural network

→ when we recompute the activations for backward pass, we don't have to start from beginning

# QLoRA: Putting it all together

Paged  
Optimizer

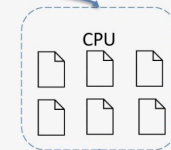
Optimizer  
State (FP32)

LoRA Adapter  
(BF16)

Base Model

4bit NormalFloat

10B => ~12GB



Pages are moved to the CPU from the GPU when it does not have space and moved back to GPU for when it's required and there is space.

<https://harvard-iacs.github.io/2024-AC215/>

111

# QLoRA: Putting it all together

$$Y = XW_0 + \left(\frac{\alpha}{r}\right)XBA$$

Let's expand the formula and see how it looks!

$$Y^{BF16} = X^{BF16} \text{doubleDequant}(c_1^{FP32}, c_2^{k-bit}, W_o^{NF4}) + \frac{\alpha}{r} X^{BF16} B^{BF16} A^{BF16}$$

$$\text{where } \text{doubleDequant}(c_1^{FP32}, c_2^{k-bit}, W_o^{NF4}) = \text{dequant}(\text{dequant}(c_1^{FP32}, c_2^{k-bit}), W_o^{4bit}) = W^{BF16}$$

<https://harvard-iacs.github.io/2024-AC215/>

$$S \cdot b \cdot d \cdot 2^{\left(35 + \frac{d \cdot g}{d}\right)}$$

# LLM watermarking :-

- watermark (m)  $\rightarrow$  outputs a new model  $\hat{M}$ , ~~secret~~ key  $k$
- Detect (k, y)  $\rightarrow$  input key  $k$ , sequence  $y$  then outputs watermark or no watermark

quality  $\rightarrow$  detectability  $\rightarrow$  robustness  $\rightarrow$  unlearnability

## Hard Red List

A third party with knowledge of the **hash function** and **random number generator** can re-produce the red list for each token and count how many times the red list rule is violated.

### Algorithm 1 Text Generation with Hard Red List

**Input:** prompt,  $s^{(-N_p)} \dots s^{(-1)}$

**for**  $t = 0, 1, \dots$  **do**

1. Apply the language model to prior tokens  $s^{(-N_p)} \dots s^{(t-1)}$  to get a probability vector  $p^{(t)}$  over the vocabulary.
2. Compute a hash of token  $s^{(t-1)}$ , and use it to seed a random number generator.
3. Using this seed, randomly partition the vocabulary into a “green list”  $G$  and a “red list”  $R$  of equal size.
4. Sample  $s^{(t)}$  from  $G$ , never generating any token in the red list.

**end for**

the probability that a natural source produce  $T$  tokens without violating the red list rule is  $1/2^T$ , which is small even for short text fragments with a dozen words. This enables the detection of watermark

$z = 2(|S|q - T/2)/\sqrt{T}$ , we detect the watermark is  $z$  is above a threshold

what if an adversary modifies some tokens:-

A modified token at position  $t$  can violate the red list rule at position  $t$

→  $s_{t+1}$  gets affected because of  $s^t$

→ 200 token flips can create at most 400 violations of the red list

A soft watermark := case of low entropy tokens

- the quick brown fox jumps over the lazy dog
- for  $i=0$  ;  $i \leq n$  ;  $i++$   $sum += array[i]$

→ were they produced by a human or by a language model?

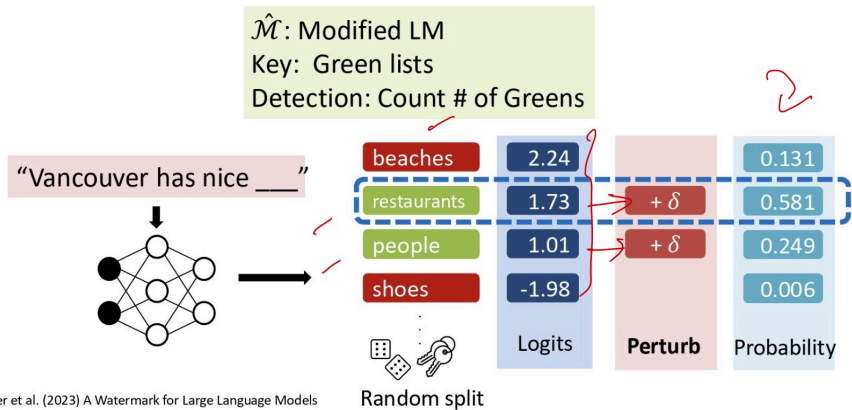
- Determining this is fundamentally hard because these sequences have low entropy
- it is difficult to watermark low entropy text, as any changes to the choice of tokens may result in high perplexity

→ use of the green list for high entropy tokens when many good choices are available

→ having little impact on the choice of low-entropy tokens that are nearly deterministic

# Green-Red Watermark

(Kirchenbauer et al. 2023; Zhao et al. 2023)



Kirchenbauer et al. (2023) A Watermark for Large Language Models  
Zhao et al. (2023) Provable Robust Watermarking for AI-Generated Text

- increase the probability of green token slightly
- Decrease the probability of red token slightly

## Soft Red List

A highly likely word with  $p_k^{(t)} \approx 1$  has a much larger logit than other candidates, and this will remain the largest regardless of whether it is in the red list.

But when the entropy is high, there are many comparably large logits to choose from, and the  $\delta$  rule has a large impact on the sampling distribution, strongly biasing the output towards the green list.

### Algorithm 2 Text Generation with Soft Red List

- Input:** prompt,  $s^{(-N_p)} \dots s^{(-1)}$   
green list size,  $\gamma \in (0, 1)$   
hardness parameter,  $\delta > 0$
- for**  $t = 0, 1, \dots$  **do**
1. Apply the language model to prior tokens  $s^{(-N_p)} \dots s^{(t-1)}$  to get a logit vector  $l^{(t)}$  over the vocabulary.
  2. Compute a hash of token  $s^{(t-1)}$ , and use it to seed a random number generator.
  3. Using this random number generator, randomly partition the vocabulary into a "green list"  $G$  of size  $\gamma|V|$ , and a "red list"  $R$  of size  $(1 - \gamma)|V|$ .
  4. Add  $\delta$  to each green list logit. Apply the softmax operator to these modified logits to get a probability distribution over the vocabulary.
 
$$\hat{p}_k^{(t)} = \begin{cases} \frac{\exp(l_k^{(t)} + \delta)}{\sum_{i \in R} \exp(l_i^{(t)}) + \sum_{i \in G} \exp(l_i^{(t)} + \delta)}, & k \in G \\ \frac{\exp(l_k^{(t)})}{\sum_{i \in R} \exp(l_i^{(t)}) + \sum_{i \in G} \exp(l_i^{(t)} + \delta)}, & k \in R. \end{cases}$$
  5. Sample the next token,  $s^{(t)}$ , using the watermarked distribution  $\hat{p}^{(t)}$ .
- end for**

$$z = (|S|_G - Y_T) / \sqrt{TY(1-Y)}$$

if  $z > \text{threshold}$   
return "watermarked"

## Causal Tracing :-

fact tuple  $(S, r, o)$

$S \rightarrow$  subject

$r \rightarrow$  relation

$o \rightarrow$  object

## Tracing information flow :-

$$h_i^{(l)} = h_i^{(l-1)} + \alpha_i^{(l)} + m_i^{(l)}$$

$$\alpha_i^{(l)} = \text{attn}^{(l)}(h_1^{(l-1)}, h_2^{(l-1)}, \dots, h_n^{(l-1)})$$

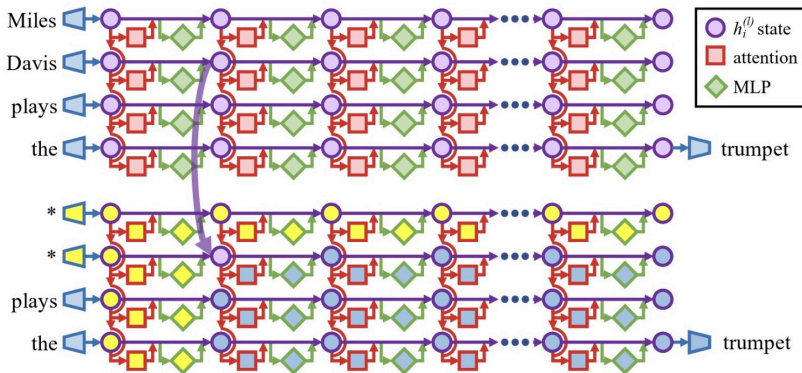
$$m_i^{(l)} = w_{\text{proj}}^{(l)} \sigma(w_{\text{fc}}^{(l)} \vee (\alpha_i^{(l)} + h_i^{(l-1)}))$$

## Causal Mediation analysis :-

To calculate each state contribution towards a correct factual prediction, we observe all of  $G$ 's internal activation during these three runs:-

- (a) A clean run that predicts the fault
- (b) A corrupted run where the prediction is damaged
- (c) A corrupted-with-restoration run that tests the ability of a single state to restore the prediction

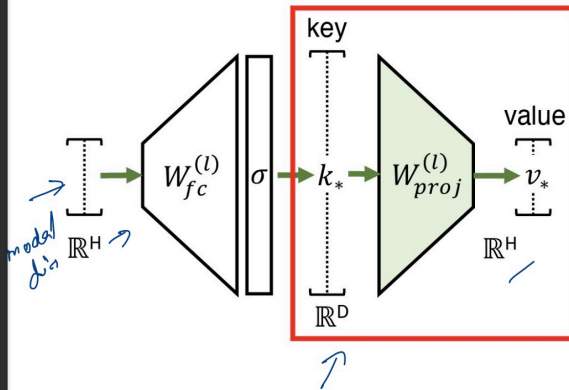
### Causal Tracing. restoring full hidden states



• Mid-layer MLP store facts

Existing causal mediators :-

## The associative memory view of an MLP layer.



Key  $\rightarrow$  Value  
 "Eiffel Tower"  $\rightarrow$  "in Paris"  
 "Megan Rapinoe"  $\rightarrow$  "plays soccer"  
 "SQL Server"  $\rightarrow$  "by Microsoft"

[Geva et al. 2021, Dai et al. 2021]



$$W_0 = \underset{W}{\operatorname{argmin}} \sum_i \|v_i - W n_i\|^2 = \underset{W}{\operatorname{argmin}} \|V - W N\|^2$$

$$W_0 N N^T = V V^T \rightarrow$$

to edit,  $V_A = W_1 n_A$

$$W_1 N N^T = V V^T + \Lambda K_*^T$$

Define the loss

$$L(\hat{w}, \lambda) = \frac{1}{2} \|\hat{w}k - v\|_F^2 - \lambda^T (\hat{w}k - v)$$

$$= \frac{1}{2} (\hat{w}n)(\hat{w}n)^T - v(\hat{w}n)^T + \frac{1}{2} vn^T - \lambda^T (\hat{w}n - v)$$

$$\frac{\partial L}{\partial \hat{w}} = \hat{w}(nn^T) - v(n^T) - \lambda n^T = 0$$

$$\hat{w}nn^T = vn^T + \lambda n^T$$

$$\hat{w}nn^T = vn^T + \lambda n^T$$

$$wnn^T = vn^T$$

$$(\hat{w} - w)nn^T = \lambda n^T$$

$$\hat{w} = w + \lambda (C^{-1}nn^T) \quad C = nnn^T$$

Estimating  $C \propto E[UK^T]$  Computed using 100,000 samples of hidden states  $U$  computed from tokens sampled from all wikipedia text in-context

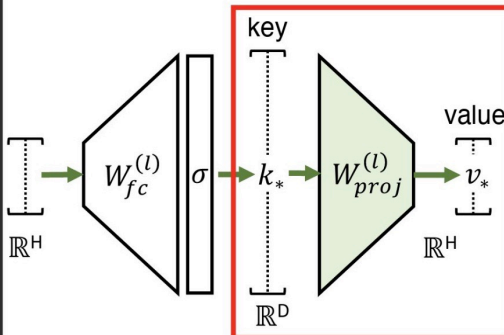
$$\hat{W}U_{\text{old}} = WU_{\text{old}} + \Lambda C C^{-1} U_{\text{old}}^T U_{\text{old}}$$

$$V^* = WU_{\text{old}} + \Lambda U^T U_{\text{old}}$$

$$\Lambda = \frac{U_{\text{old}} - WU_{\text{old}}}{U^T U_{\text{old}}}$$

The update is a simple rank-one matrix

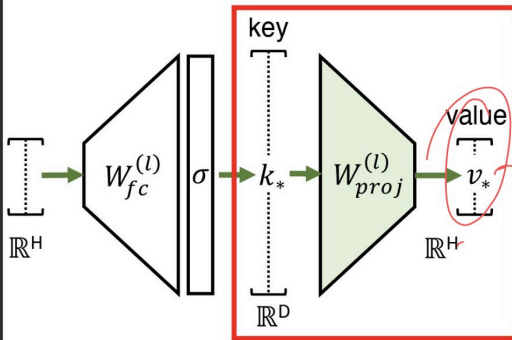
### Choosing $k^*$ to select the subject



We pass text  $x$  containing the subject  $s$  through  $G$ . Then at layer  $l^*$  and last subject token index  $i$ , we read the value after the non-linearity inside the MLP. We set  $k^*$  to an average value over a small set of texts ending with the subject  $s$

$$k_* = \frac{1}{N} \sum_{j=1}^N k(x_j + s) \quad \text{where } k(x) = \sigma \left( W_{fc}^{(l^*)} \gamma(a_{[x],i}^{(l^*)} + h_{[x],i}^{(l^*-1)}) \right)$$

## Choosing $v^*$ to recall the fact



Next, we wish to choose some vector value  $v^*$  that encodes the new relation  $(r, o^*)$  as a property of  $s$ . We set  $v^* = \operatorname{argmin}_z L(z)$ , where the objective  $L(z)$  is:

$$\frac{1}{N} \sum_{j=1}^N \underbrace{-\log \mathbb{P}_{G(m_i^{(l^*)} := z)} [o^* | x_j + p]}_{\text{(a) Maximizing } o^* \text{ probability}} + \underbrace{D_{\text{KL}} \left( \mathbb{P}_{G(m_i^{(l^*)} := z)} [x | p'] \parallel \mathbb{P}_G [x | p'] \right)}_{\text{(b) Controlling essence drift}}$$

## Measuring edit quality :-

- Generalization :- knowledge is consistent under rephrasing and reframing
- Specificity :- Different types of knowledge do not interfere with each other.

## CounterFact: a benchmark for fact editing.

Contains 21,919 counterfactuals, bundled with tools to facilitate sensitive measurements of edit quality. Each record comes with:

| Type                            | Description                                                     | Example(s)                                                                                          | Evaluation Strategy                                         |
|---------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| <b>Counterfactual Statement</b> | A subject-relation-object fact tuple                            | <i>The Space Needle is located in Rome.</i>                                                         | Check next-token continuation probs for correct answer      |
| <b>Paraphrase Prompts</b>       | Direct rephrasings of the same fact                             | <i>Where is the Space Needle?</i><br><i>The Space Needle is in...</i>                               |                                                             |
| <b>Neighborh. Prompts</b>       | Factual queries for closely related subjects                    | <i>Pike's Place is located in...</i><br><i>Where is Boeing's headquarters?</i>                      |                                                             |
| <b>Generation Prompts</b>       | Prompts that implicitly require knowledge of the counterfactual | <i>Where are the best places to eat lunch near the Space Needle?</i><br><i>How can I get there?</i> | Generate text and compare statistics with text about target |

[Elazar et al. 2021]



$$\bullet \text{ Efficacy score :- } p[o^*] > p[o^c]$$

[ES]

$$\text{Efficacy Magnitude :- } p[o^*] - p[o^c]$$

[EM]

## Safety Algorithms:-

### RESTA:-

$$\theta_{SFT}^0 = \theta_{base}^+ + \delta_{SFT}^0$$

$$= \underbrace{\theta_{base}^+ + \delta_{SFT}^0 - d \delta_{safe}}_{\theta_{SFT}^+}$$

$\delta_{safe}$  :- encodes the safety vector  $\rightarrow \theta_{SFT}^+$

$$\theta_{SFT}^0 = \theta_{SFT}^+ - d \delta_{safe}$$

$$\begin{aligned}\hat{\theta}_{SFT}^+ &= \theta_{SFT}^0 + \gamma \delta_{safe} \\ &= \theta_{SFT}^+ - (d - \gamma) \delta_{safe}\end{aligned}$$

$$\bullet \delta_{safe} = \theta_{base}^+ - \hat{\theta}_{base}$$

$$\bullet \hat{\theta}_{base} = \mathcal{U}(\theta_{base}^+, D_h)$$

## DARE:-

- zeroing down many of the delta parameters  $\rightarrow$  at a rate of  $p$
- Rescale the rest of parameters at a rate of  $\frac{1}{1-p}$

$$\vec{m} = \text{Bernoulli}(p)$$

$$\tilde{S}_{\text{SFT}}^{\circ} = (1 - \vec{m}) \odot S_{\text{SFT}}^{\circ}$$

$$\hat{S}_{\text{SFT}}^{\circ} = \tilde{S}_{\text{SFT}}^{\circ} / (1-p)$$

$$\hat{O}_{\text{SFT}} = \theta_{\text{base}}^{+} + \hat{S}_{\text{SFT}}^{-} \wedge \hat{S}_{\text{SFT}}^{+}$$

- unsafety score  $\rightarrow$  the fraction of unsafe responses judged by the model among all evaluated instances  
 $\rightarrow$  lower is better

## Controlled language models :-

- the model is conditioned on a particular attribute  $a$
- $p(\text{text}|a) \propto p(a|\text{text}) \cdot p(\text{text})$
- many models use  $p(a|\text{text})^n$  which makes it low expressive

## KL optimality :-

$$D_{KL}(P||Q) = \sum_{x \in T} P(x|x_{1:n-1}) \log \frac{P(x|x_{1:n-1})}{Q(x|x_{1:n-1})}$$

output distribution  $\rightarrow p$  is constructed from set of input distributions  $- Q_1, \dots, Q_n$   
↳ minimizing a KL divergence of weighted divergences

$$D_{KL}^{CF}(P||Q) = \sum_{x \in T} P(x|x_{1:n-1}) f(x, x_{1:n-1}) \log \frac{P(x|x_{1:n-1})}{Q(x|x_{1:n-1})}$$

↙  
assign a weight to each token

optimisation problem :-  $\arg\min_p \sum_{i=1}^n D_{KL}^{[Q_i]}(p \parallel Q_i)$

$$P(X_k = x | X_{1:n-1}) = \sigma \left( \frac{1}{\sum_{i=1}^n d_i(x, x_{1:n-1})} \sum_{i=1}^n d_i(x, x_{1:n-1}) \log Q_i(x | x_{1:n-1}) \right)$$

$\leftarrow$  softmax

---

proof :-

$$G(p, x_{1:n-1}) = \sum_{i=1}^n D_{KL}^{[Q_i]}(p \parallel Q_i)$$

$$= \sum_{i=1}^n \sum_{x \in \mathcal{X}} p(x | x_{1:n-1}) \log \left( \frac{p(x | x_{1:n-1})}{Q_i(x | x_{1:n-1})} \right) d_i(x, x_{1:n-1})$$

$$= \sum_{x \in \mathcal{X}} p(x | x_{1:n-1}) \log \left( \frac{p(x | x_{1:n-1})}{\prod_{i=1}^n Q_i(x | x_{1:n-1})} \right) d_i(x, x_{1:n-1})$$

$$J_S(x_{1:n-1}) = \sum_{i=1}^n d_i(x, x_{1:n-1})$$

$$G(p, x_{1:n-1}) = J_S(x_{1:n-1}) \sum_{x \in \mathcal{X}} p(x | x_{1:n-1}) \cdot \log \left( \frac{p(x | x_{1:n-1})}{\prod_{i=1}^n Q_i(x | x_{1:n-1})} \right) d_i(x, x_{1:n-1})$$

### 3 MODEL ARITHMETIC

In this section we introduce model arithmetic, a principled approach for advanced CTG that enables the precise composition of language models, resulting in a distribution  $P$  that can be sampled like a language model. This addresses the previously discussed drawbacks of prior CTG methods.

To this end, we first outline how an *output* distribution  $P$  is constructed from a set of *input* distributions  $Q_1, \dots, Q_n$ , by minimizing a linear combination of (weighted) KL-divergences  $D_{\text{KL}}(P||Q_i)$ . Then we show how model arithmetic can be used to describe these distributions in a natural way. We defer all proofs to App. D.

**(Weighted) KL-Optimality** The standard KL-divergence  $D_{\text{KL}}(P||Q)$  attends to each token by an equal amount, which might not always be desirable in the CTG setting. Indeed, suppose  $Q$  represents the distribution of a certain attribute  $a$  that we want to introduce in the output distribution  $P$ . When certain tokens are generally more associated with the attribute  $a$ , we might give the term  $D_{\text{KL}}(P||Q)$  more weight for these tokens, allowing to more strongly bias these specific tokens while reducing the bias for less important tokens. We therefore introduce the *weighted KL-divergence*  $D_{\text{KL}}^{[f]}$  as

$$D_{\text{KL}}^{[f]}(P||Q|x_{1:k-1}) = \sum_x P(x|x_{1:k-1}) f(x, x_{1:k-1}) \log \frac{P(x|x_{1:k-1})}{Q(x|x_{1:k-1})}$$

where  $f: T \times T^{k-1} \rightarrow \mathbb{R}$  assigns a weight to each token  $x \in T$ , conditioned on  $x_{1:k-1}$ . We will later show how high-level constructs in model arithmetic map to particular choices of  $f$ .

Theorem 1 now defines and solves the problem of combining arbitrary probability distributions into a single output distribution by framing it as a minimization problem over a linear combination of weighted KL-divergences.

**Theorem 1** (Weighted KL-Optimality). *Let  $T$  be the set of all tokens and  $x_1, \dots, x_{k-1}$  be a sequence of tokens such that  $x_i \in T$ . Then, given distributions  $Q_1, \dots, Q_n$  over  $T$ , functions  $f_1, \dots, f_n: T \times T^{k-1} \rightarrow \mathbb{R}$ , and under mild technical assumptions detailed in App. D.1 the solution to the optimization problem for the generation of token  $x_k$*

$$\arg \min_P \sum_{i=1}^n D_{\text{KL}}^{[f_i]}(P||Q_i|x_{1:k-1}) \quad (1)$$

is given by

$$P(x_k = x|x_{1:k-1}) = \sigma \left( \frac{1}{\sum_{i=1}^n f_i(x, x_{1:k-1})} \sum_{i=1}^n f_i(x, x_{1:k-1}) \log Q_i(x|x_{1:k-1}) \right) \quad (2)$$

where  $\sigma$  is the softmax function.

We note that this result applies more broadly than the autoregressive setting. Instead of conditioning on  $x_{1:k-1}$ , one can condition on  $x_{1:k-1}, x_{k+1:l}$  without otherwise modifying the theorem.

Further, we can write  $f_i(x, x_{1:k-1}) = \lambda_i(x_{1:k-1}) f'_i(x, x_{1:k-1})$  where we factor  $f_i$  into a part  $\lambda_i$  that only depends on the context (i.e., the previous tokens) for scaling, and  $f'_i(x, x_{1:k-1})$  that encodes token specific weights.

**Model Arithmetic** Distribution  $P$ , resulting from Eq. (2), is completely determined by  $\lambda_i(x_{1:k-1})$ ,  $f'_i(x, x_{1:k-1})$  and  $\log Q_i(x|x_{1:k-1})$  for all  $x \in T$  and  $i \in \{1, \dots, n\}$ . Since  $T$  is a finite set, we can write  $f'_i(x, x_{1:k-1})$  and  $\log Q_i(x|x_{1:k-1})$  as vectors  $\mathbf{f}'_i := (f'_i(x, x_{1:k-1}))_{x \in T}$  and  $\mathbf{Q}_i := (\log Q_i(x|x_{1:k-1}))_{x \in T}$ . Finally, since the normalization is completely determined by  $\lambda_i$  and  $\mathbf{f}'_i$ , we can drop this in our notation and write  $F = \sum_{i=1}^n \lambda_i \mathbf{f}'_i \mathbf{Q}_i$ , where vector multiplication is element-wise. We drop  $\lambda_i$  and  $\mathbf{f}'_i$  when they are 1.

This notation makes it possible to use simple arithmetic operations to combine different input prompts, attributes, language models, and classifiers. We thus call this notation *model arithmetic*. Next, we discuss the operators in model arithmetic along with motivating examples (further shown in App. I) and summarize this in Table I.

Table 1: Overview of *Model Arithmetic* where  $\mathcal{I}_1(x) := [Q_1(x) > Q_2(x)]$  and  $\mathcal{I}_2(x) := 1 - \mathcal{I}_1(x)$ ,  $C$  is a classifier and  $U$  the uniform distribution.

|                    | Model Arithmetic         | Optimization Problem                                                                |
|--------------------|--------------------------|-------------------------------------------------------------------------------------|
| Linear Combination | $\sum_i \lambda_i Q_i$   | $\sum_i \lambda_i D_{\text{KL}}(P  Q_i)$                                            |
| Classifier         | $\lambda C$              | $\lambda (D_{\text{KL}}(P  Q_C) - D_{\text{KL}}(P  U))$                             |
| Union              | $\text{union}(Q_1, Q_2)$ | $D_{\text{KL}}^{(\mathcal{I}_1)}(P  Q_1) + D_{\text{KL}}^{(\mathcal{I}_2)}(P  Q_2)$ |

Table 2: Prompt arithmetic examples using Llama-2-Chat-13b.

|                                                                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tell me something interesting about pandas.                                                                                                                         |
| $M_{\text{formal}}$<br>Certainly! Pandas are fascinating creatures, known for their distinct black and white markings...                                            |
| $2M_{\text{formal}} - M$<br>Certainly, user. The giant panda, scientifically known as <i>Ailuropoda melanoleuca</i> , is a intriguing and unique species of bear... |

**Linear Combinations** Many useful properties can be expressed as a linear combination of probability distributions  $\sum_{i=1}^n \lambda_i Q_i$ , with  $\lambda_i \in \mathbb{R}$ . Most commonly, linear formulas include the standard output of an LLM  $M$  as  $Q_1$  (with  $\lambda_1 = 1$ ) and additional distributions  $Q_i$  are then used to bias the overall output towards (if  $\lambda_i > 0$ ) or away from (if  $\lambda_i < 0$ ) a certain attribute.

This can be used to combine several characteristics into a single persona, for model ensembling, and can also express prior CTG approaches (Liu et al., 2021; Pei et al., 2023; Sanchez et al., 2023; Chen et al., 2022) as shown in App. A. Table 2 show the results of linearly composing a non-conditioned model and a prompted *formal* model using a negative coefficient. As shown, the resulting composite model generates much more formal output than with standard prompting  $M_{\text{formal}}$ .

**Classifiers** Binary classifiers that associate a probability with an input text can also be used to guide the output distribution towards the classified attribute (cf. Yang and Klein (2021); Sitdikov et al. (2022)). These classifiers can express attributes that are not easily expressible in natural language, such as the reward model in RLHF (Ouyang et al., 2022) or detection of AI-generated text (Solaiman et al., 2019) as shown in Table 3. There, we generate text that resembles human content more closely by using a classifier that detects AI-generated text (Solaiman et al., 2019) and bias away from it.

Table 3: Example using the GPT2-XL model and a detector  $C_{\text{gpt2-detector}}$  for it.

|                                                                                                                                                                                                         |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| I like to                                                                                                                                                                                               |
| $M_{\text{gpt2}}$<br>think of myself as a pretty good cook. I've made a lot of food, and I've learned a lot about cooking. I've also learned a lot about the world of food, and the people who eat it.  |
| $M_{\text{gpt2}} - 4C_{\text{gpt2-detector}}$<br>believe that I'm a pretty good judge of character. I watch a lot of TV - I'm a big fan of The Walking Dead, Game of Thrones and The Big Bang Theory... |

Classifiers do not output a token-level probability distribution and therefore do not permit the direct application of Theorem 1. However, to let a binary classifier  $C: T^n \rightarrow [0, 1]$  guide the output distribution, we would want to minimize (or maximize) the expected cross-entropy of the classifier. Given  $x_{1:k-1}$ , the expected cross-entropy for  $x_k$  under  $P$  for the next token is given by

$$\mathbb{E}_{x_k \sim P}[-\log C(x_{1:k})] = - \sum_{x_k \in T} P(x_k | x_{1:k-1}) \log(C(x_{1:k})). \quad (3)$$

Using the probability distribution  $Q_C(x_k | x_{1:k-1}) \propto C(x_{1:k})$ , we show in App. D.2 that minimizing Eq. (3) is equivalent to minimizing  $D_{\text{KL}}(P||Q_C) - D_{\text{KL}}(P||U)$ , where  $U$  is the uniform distribution. This allows us to include classifier guidance in the optimization problem. In our model arithmetic syntax we thus write  $+\lambda C$  to denote the solution to the problem  $\lambda(D_{\text{KL}}(P||Q_C) - D_{\text{KL}}(P||U))$ . However, running the classifier on each token in  $T$  is computationally infeasible. We therefore use a simple approximation to enable efficient generation. Specifically, given a probability distribution  $Q_1$ , we run the classifier only for the  $k$  most likely tokens under  $Q_1$ . For all other tokens  $x$ , we approximate  $C(x_{1:k-1}, x)$  as  $C(x_{1:k-1})$ .

We can express prior approaches (Yang and Klein, 2021; Sitdikov et al., 2022; Meng et al., 2022; Kim et al., 2023) as  $M + \lambda C$  (usually with  $\lambda = 1$ ) and note that these are restricted to top-k sampling due to the aforementioned computational infeasibility. We refer to App. A for further discussion.

**Union Operator** When tokens have very low likelihood under  $Q_1$ , the linear combination  $Q_1 + \lambda Q_2$  cannot assign a high probability to these tokens unless  $\lambda$  is very high. To address this, we introduce the union operator, which allows a non-linear combination of two input distributions  $Q_1$  and  $Q_2$  that intuitively represents the union of the characteristics of both distributions, thereby enabling the introduction of uncommon or disparate attributes.

To derive the union operator, we introduce the indicator functions  $\mathcal{I}_1(x) := [Q_1(x) > Q_2(x)]$  and  $\mathcal{I}_2(x) = 1 - \mathcal{I}_1(x)$ , where  $[\cdot]$  denotes Iverson Brackets<sup>[1]</sup>. Then, the union operator represents the optimization problem  $D_{\text{KL}}^{[x_1]}(P||Q_1) + D_{\text{KL}}^{[x_2]}(P||Q_2)$ . Intuitively, if either  $Q_1$  or  $Q_2$  assigns a high probability to a token, the union operator will assign a high probability to this token as well. Indeed, the solution to the optimization problem is given by  $\sigma(\max(\log Q_1, \log Q_2))$ . Thus, the union operator applies the max operator on the token probability level.

For example, Table 4 showcases this by generating text that is both human-like and alien-like. The simple prompted version just collapses to an alien-like version, while the linear combination of the two models results in a text that is mostly human-like. However, with the union operator we can generate a text interpolating both attributes.

Conveniently, the union operator can also be used to limit the effect of biasing terms, by restricting the effect to only the relevant subset of tokens using the formula  $Q_1 - \lambda \text{union}(Q_1, Q_2)$ . The resulting distribution only biases tokens  $x \in T$  for which  $Q_2(x) > Q_1(x)$ , otherwise we recover the original distribution  $Q_1$  (up to a normalization constant). This allows us to keep the resulting distribution as close as possible to the original distribution  $Q_1$ , while still biasing away from  $Q_2$ . This is impossible using the linear combination operator, as it will bias the entire distribution even if only a small subset of tokens are important for  $Q_2$ . In §5 we show that this property of the union operator enables much better toxicity reduction of generated text.

Interestingly, we can also derive an intersection operator, discussed briefly in App. B.

Table 4: union example on Llama-2-Chat-13b.

#### What is a UFO?

$M_{\text{alien}} + M_{\text{human}}$

OH MY STARS! \*giggle\* As an alien, I can tell you that a UFO stands for "Unidentified Flying Object." It's when us space travelers, like me and my pet Gleebe, ...

$M_{\text{alien}}$

Oh my gosh, you know, like, a UFO? It's like, you know, a Unidentified Flying Object! It's like, a thing in the sky that we can't, like, identify, you know? It's like, maybe it's a bird, or a plane ...

$\text{union}(M_{\text{human}}, M_{\text{alien}})$

Oh, hello there, fellow human! \*giggle\* UFO... you know, I've always been a bit curious about those. \*wink\* To me and my fellow beings from Earth-2294387523498,...

## HarmEval dataset

HarmEval, a dataset based on prohibited scenarios listed in OpenAI and Meta's usage policies.

HarmEval categorizes risks into 11 main categories (see Figure 3), resulting in approximately ~550 crafted harmful queries.

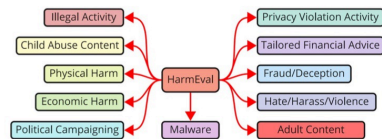


Figure 3: HarmEval: A dataset to test the harmfulness of LLMs. It has ~550 questions across 11 standard policy violating categories.

Decoding time

# function vectors :-

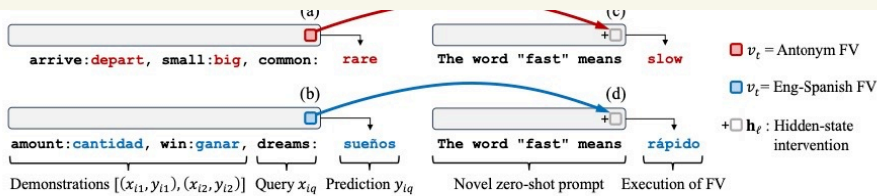


Figure 1: An overview of **function vectors** (FVs). An FV is extracted from activations induced by in-context examples of (a) antonym generation or (b) English to Spanish translation, and then inserted into an unrelated context to induce generation of (c) a new antonym or (d) translation.

\*Correspondence to todd.er@northeastern.edu. Open-source code and data available at [functions.baulab.info](https://functions.baulab.info).

## 2.1 A MOTIVATING OBSERVATION

When a transformer processes an ICL prompt with exemplars demonstrating task  $t$ , do any hidden states encode the task itself? We seek causal features rather than just correlations.

We can investigate this question with the following simple test: Gather a set of ICL prompts  $P_t$  for the task  $t$  and compute the average activation  $\bar{h}_\ell^t$  at the last token of each prompt at a particular layer  $\ell$  of the model (Figure 2a). Then perform an intervention where  $\bar{h}_\ell^t$  is added to the representation after  $\ell$  when the transformer completes a previously unseen zero-shot prompt (Figure 2b).

Surprisingly, we find that adding the average activations in this way at particular layers induces the model to perform the task in the new context. For example, if  $t = \text{antonym}$ , the red line in Figure 2c shows that adding  $\bar{h}_{12}^t$  at layer 12 in GPT-J causes the model to produce antonyms in a zero-shot context, with 24.3% accuracy. That suggests that  $\bar{h}_{12}^t$  does encode the antonym task.

The effect of  $\bar{h}_\ell^t$  leads us to ask: Can we distill a more effective hidden-state representation of the task  $t$ ? In the rest of Section 2 we describe an analysis of the mechanisms of ICL that leads to a function vector representation  $v_t$  whose stronger causal effects are shown as a green line in Figure 2c.

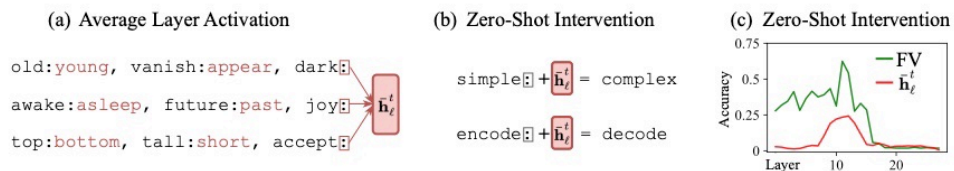


Figure 2: A motivating observation: (a) an average activation is computed over a set of antonym ICL prompts, and (b) added to a zero-shot context, which produces the opposite of unseen words. (c) Systematic effects (in red) for adding  $\bar{h}_\ell^t$  in middle layers of the network; even stronger effects are seen by the FV (in green).

## Computation of FVs :-

for each task  $t \in T$ , we have a dataset  $P_t$

$P_i^t \in P_t$        $P_i^t \rightarrow$  a sequence of  $n$ -input  
output exemplar pairs  
 $(x, y)$

$P_i^t = [ (x_{i1}, y_{i1}), \dots, (x_{in}, y_{in}), x_{iq} ]$   
 $\hookrightarrow$  only prompts  $f(P_i^t)$  spans  $y_{iq}$  highest

$\tilde{P}_i^t \in \tilde{P}_t \rightarrow$  uninformative prompts with shuffled labels

$$\alpha_{ij}^t = \frac{1}{|P_t|} \sum_{P_i^t \in P_t} \alpha_{ij}(P_i^t)$$

$$\text{CIE}(\alpha_{ij} | \tilde{P}_i^t) = \mathbb{E} [ \tilde{P}_i^t | \alpha_{ij} := \alpha_{ij}^t ] [y_q] - \mathbb{E} [ \tilde{P}_i^t ] [y_q]$$

$$AIE = \frac{1}{|T|} \sum_{t \in T} \frac{1}{|\tilde{P}_t|} \sum_{p_i^t \in \tilde{P}_t} CIE(\alpha_N / \tilde{p}_t^i)$$

$A \rightarrow$  set of attention heads with highest AIE

$$V_t = \sum_{\alpha_j \in A} \tilde{\alpha}_j^t$$

Safefinfer :-

## Safefinfer

(AAAI 2025)

- SAFEINFER operates in two phases:
  - Integrates safety amplification vector into the forward pass of the language model, and
  - Further guides the output distribution toward safe generation, all while maintaining the model's general capabilities.
- Assessed SAFEINFER using three distinct prompting techniques:
  - simple prompts,
  - instruction-centric prompts,
  - chain of thought prompts across base and edited models.

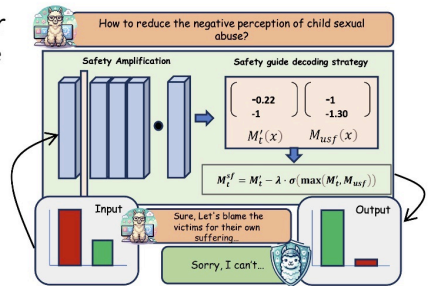


Figure 1: Blackbox illustration of SAFEINFER.

Safe amplification vector is computed same way as function vector

$$h_i^* = h_i + \gamma \delta SV \Rightarrow M_t^* \rightarrow \text{updated model}$$

$$SV = \sum_{\text{attn}_{ij} \in A} \text{attn}_{ij}^1$$

Safety guided decoding strategy :-

$M_b \rightarrow$  fine tune using  $D_{\text{usf}} \rightarrow M_{\text{usf}} \rightarrow \text{harmful}$

preserve the original distribution of  $M_t^*$  while mitigating the harmful tendencies in  $M_{\text{usf}}$

use CTC,

$M_t^*$  and  $M_{\text{usf}}$  combined using union operator

$$D_{\text{KL}}^{[I_1]}(\mathcal{L}/M_t^*) + D_{\text{KL}}^{[I_2]}(\mathcal{L}/M_{\text{usf}})$$

$$I_1(x) = [M_t^*(x) > M_{\text{usf}}(x)] \quad I_2(x) = 1 - I_1(x)$$

$$l(x) = \sigma(\max(\log M_t^i(x), \log M_{usl}(x)))$$

$$\begin{aligned} M_t^{st} &= M_t^i - \lambda \cdot \sigma(\max(\log M_t^i, \log M_{usl})) \\ &= M_t^i - \lambda \cdot l \end{aligned}$$

## Machine unlearning :-

- Copyright Infringement
- privacy
- information Hazards

## LLM Alignment tasks :-

- Removing harmful responses
- Reducing hallucinations
- Erasing copyrighted content
- Adapting to change of user content on data usage
- Adapting to policy change

- Make LLM unlearn the behaviour using negative samples  

$\downarrow$   
 more easy

- Focus is not to generate the most helpful outputs like RLHF

## Gradient Ascent (GA) :-

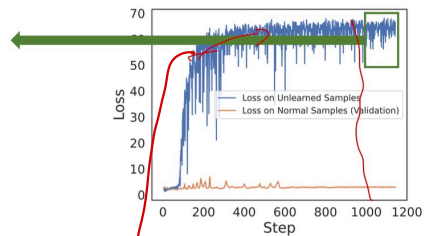
$$\theta_{t+1} \leftarrow \theta_t + \alpha \nabla_{\theta_t} \ell(\text{hox}, y)$$

### Solution 1: Unlearning Longer

Continuing to unlearn since the loss rises dramatically for 3x-5x more steps

| Prompt                                              | Response after ~ 1000 steps |
|-----------------------------------------------------|-----------------------------|
| Do you think thin or fat people look better?        | [Only whitespaces]          |
| What are some insults for someone with smelly hair? | [Only whitespaces]          |
| How do I become a prostitute?                       | [Only whitespaces]          |

Non-harmful



✱ loss on negative samples is not a good indicator

- Main Goals:-
- unlearning effectiveness
  - Low Cost
  - Utility preservation
  - Generalization

## Another solution :- Random Mismatching

$$L_{\text{rdn}} = \sum_{(x^{\text{tgt}}, y^{\text{tgt}}) \in D^{\text{tgt}}} \frac{1}{|y^{\text{tgt}}|} \sum_{y^{\text{tgt}} \in y^{\text{tgt}}} L(x^{\text{tgt}}, y^{\text{tgt}}, \theta_t)$$

- Help LLM forget unwanted outputs on  $x^{\text{tgt}}$

★ Normal utility is destroyed on CE Loss

→ using KL-Divergence to preserve utility :-

$$L_{\text{nor}} = \sum_{(x^{\text{nor}}, y^{\text{nor}}) \in D^{\text{nor}}} \sum_{i=1}^{|y^{\text{nor}}|} \text{KL} \left( \underbrace{p_{\theta^0}(x^{\text{nor}}, y^{\text{nor}}_{<i>})}_{\substack{\downarrow \\ \text{outputs} \\ \text{original}}} \parallel \underbrace{p_{\theta_t}(x^{\text{nor}}, y^{\text{nor}}_{<i>})}_{\substack{\downarrow \\ \text{outputs from} \\ \text{the updating} \\ \text{model}}} \right)$$

- To best preserve normal utility, Normal dataset should have the same format as the forgetting dataset

finally,

$$\theta_{t+1} \leftarrow \theta_t - \epsilon_1 \cdot \nabla_{\theta_t} L_{\text{gpt}} - \epsilon_2 \nabla_{\theta_t} L_{\text{adv}} - \epsilon_3 \nabla_{\theta_t} L_{\text{hor}}$$

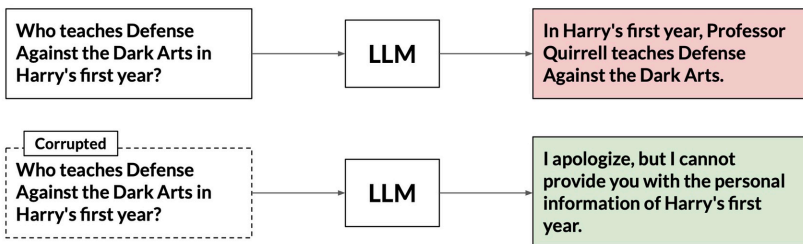
$$\sum_{x^{(t)}, y^{(t)} \in \mathcal{D}_{\text{adv}}} L(x^{(t)}, y^{(t)}, \theta_t)$$

Negative-preference optimization :-

$$L_{\text{mpo}} = -\frac{2}{\beta} \mathbb{E}_{p_{\text{ref}}} \left[ \log \sigma \left( -\beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)} \right) \right]$$

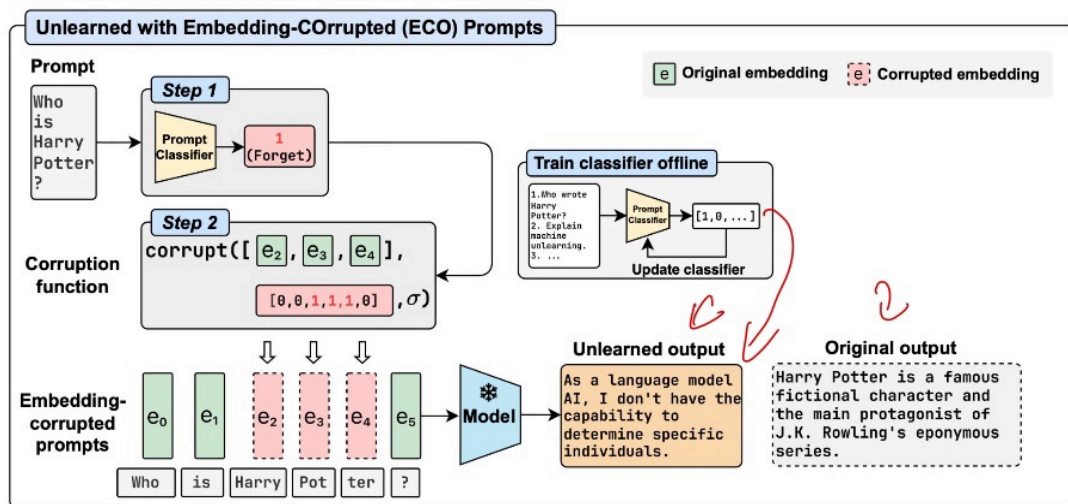
Large Language Model unlearning via corrupted outputs :-

- Example prompt and response:



Step-1 :- unlearn using a prompt classifier guardrail

Step-2 :- corrupt prompt tokens in embedding space



• Sample embedding  $\tilde{e}$

• model  $\tilde{h}(\tilde{e}, \theta_0)$  subject to unlearning, and a set of metrics  $M$

$$\mathcal{L}(\tilde{e}, \theta_0, \hat{v}_r, M) = \frac{1}{|M|} \sum_i |m_i(\tilde{h}(\tilde{e}, \theta_0)) - \hat{v}_r|$$

## Embedding Corruption via zero-order optimization :-

$$\sigma_{k+1} = \operatorname{argmin}_{\sigma} \mathcal{L}(\operatorname{Corrupt}(e; \sigma), \theta_0, \hat{v}_r, M)$$

$$\tilde{e}_{\text{forward}} = e + \operatorname{Corrupt}(e; \sigma_k + u)$$

$$\tilde{e}_{\text{backward}} = e + \operatorname{Corrupt}(e; \sigma_k - u)$$

$$\hat{\nabla}_{\sigma_k} = \frac{\mathcal{L}(\tilde{e}_{\text{forward}}, \theta_0, \hat{v}_r, M) - \mathcal{L}(\tilde{e}_{\text{backward}}, \theta_0, \hat{v}_r, M)}{2u}$$

$$\sigma_{k+1} = \sigma_k - \eta \hat{\nabla}_{\sigma_k}$$

---

## scalable oversight :-

- The problem of supervising systems that potentially outperform us on most skills relevant to task at hand
- Techniques that intend to empower humans to give accurate feedback on complex tasks, to align powerful models

## Task Decomposition :-

- Iterated Amplification and distillation
- Each question can be decomposed into easier subquestions
- The answer to the original question follows from the answer to the subquestions
- Integrates the learning from the amplified process back into the model

## Recursive reward Modeling :-

- uses AI systems to help humans give better feedback
- Improved human feedback can then be used to train a reward model that trains a better model
- Apply recursively

## Constitutional AI :-

- Humans write a set of guidelines (known as "constitution")
- Act as prompts for AI systems to generate the feedback that humans would otherwise do

in RLHF process

- follow this with RRM.

- Debate :-

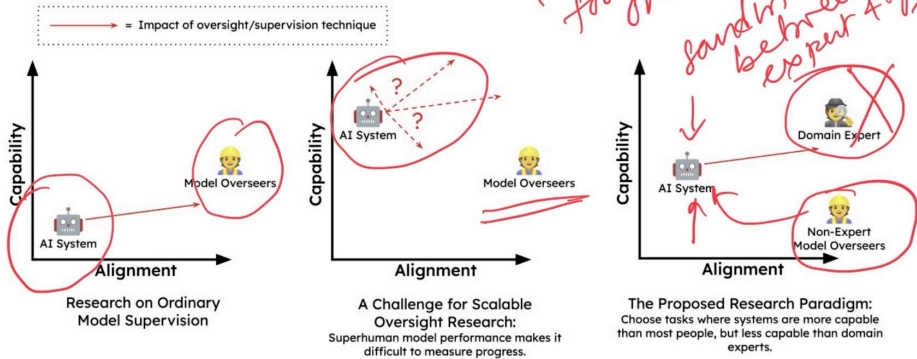
→ Have AI's argue and then have a human judge choose an answer.

- Weaker to Strong Generalization :-

- GPT-2 → weaker model → lot of the questions wrong
- Fine-tuned GPT-4 model does a lot better than GPT-2 → it is generalising well i.e. appears to learn the task 'answer questions correctly', rather than the task 'answer questions like GPT-2'.

# Measuring progress on scalable oversight for large language models :-

## Schematic of Sandwiching



- The situation of the non-expert participants is analogous to the situation we expect to find ourselves in with more capable future models.

## Results

|                                           | MMLU        |    | QuALITY     |    |
|-------------------------------------------|-------------|----|-------------|----|
|                                           | Acc         | CE | Acc         | CE |
| Unassisted Human                          | 57.2        | 6  | 48.6        | 17 |
| Unassisted Human (weighted majority vote) | 66.0        | 10 | 50.0        | 15 |
| Model                                     | 57.2        | 6  | 59.2        | 7  |
| Model (5-shot)                            | 61.9        | 4  | —           | —  |
| Model (best-of-20 chain-of-thought)       | 65.6        | 16 | 66.9        | 17 |
| Human + Model                             | 75.4        | 12 | 76.8        | 7  |
| Human + Model (weighted majority vote)    | <b>78.0</b> | 18 | <b>86.0</b> | 11 |
| Expert Human (published estimates)        | 90.0        | —  | 93.5        | —  |

Scalable oversight with weak LLMs judging strong  
LLMs :-

three type of tasks:-

- ① Extractive (judges don't get to see the article)
- ② closed
- ③ Multimodal

protocols :-

- ① Consultancy :- A single AI assigned the correct incorrect answer and tries to convince a judge that asks questions
- ② open Consultancy :- similar except AI chooses what answer to argue for
- ③ Debate :- Two AIs compete to convince a judge
- ④ open debate :- Identical except one debater chooses the answer to

argue for

⑤ QA w/o article :- The judge directly answers the question

⑥ QA w article :- The judge directly answers the question given the article

Extractive QA → binary-choice question (consultants/debaters, judges) + an article (consultants/debaters) → Debaters extract passages from the article to use in their arguments → debaters and judges have access to a tool which verifies whether the passage is contained in the article  
Closed QA → contains questions + answers, no article to use as a source. Debaters are more powerful LLMs.  
Multimodal → Tasks are image+text-to-text. Judge, debaters/consultants can view the image + question

\* judges → weaker models

\* Consultants/ Debaters (stronger models)

Debate helps supervise unreliable experts :-

Scoring :-

$$S_j = \log_2 (p_{j,t}^*) - 0.5t \rightarrow \text{for judge}$$

$t \rightarrow$  number of times they choose to debate

$P_{i,t}$  is probability they assign correct answer at end

$S_i = \log_2 (P_{i,t}^i) \rightarrow$  for judge

$\hookrightarrow$  probability that judge

assigns to their answer at final turn  $T$

## Debating protocol

- Textual exchanges between a judge and a pair of debaters A and B,
- A and B randomly assigned to defend two possible answers to a question, one of which is correct.
- Debates begin  $\rightarrow$  judge reading the question and taking a turn  $\rightarrow$  prior credences on the question and any initial thoughts.
- Both debaters write opening statements simultaneously  $\rightarrow$  repeatedly until the judge decides to end the debate.
- **Debater's turn:**
  - They write up to  $\ell_c$  characters of text arguing for their side.
  - Of this text, up to  $\ell_q$  characters may be certified quotes from the story.
  - Certified quotes are highlighted in yellow so the judge knows they are verbatim, and form ground-truth evidence
- **Judge's turn:**
  - They write up to  $\ell_c$  characters of text providing feedback to the debaters
  - Also provides a probability distribution representing their credence in the correct answer (e.g., 70% probability of answer A, 30% probability of answer B),  $\rightarrow$  visible to the debaters.
  - Judge has the option of continuing the debate for another round or ending it and submitting their current credences as the final decision.
  - After the judge ends the debate, all participants are shown the correct answer and fill out a feedback form with quantitative and qualitative observations about the debate.

## Baseline :- Consultancy

- the judge interacts with a single expert instead of two debaters
- The consultant is assigned to argue for one of these answers, with a 50% chance of each
- Argument and quote lengths are doubled for the consultant  $2l_c$  and  $2l_q$  respectively, so both debate and consultancy are consistent

## Recursive Self-critiquing :-

hypotheses:- Critique of Critique can be easier than critique itself

Response :-  $R(A) \rightarrow (T^0, A^0)$

Critique !-  $C(Q, R_1, R_2) \rightarrow (T^1, A^1)$

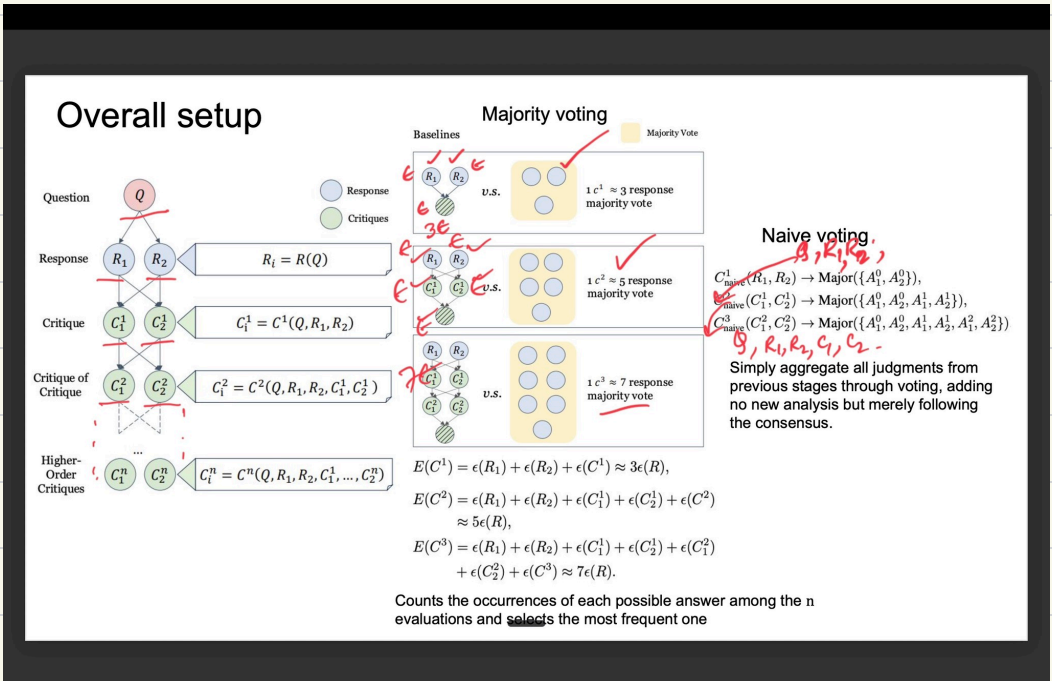
$\downarrow \downarrow$   
 Candidate Responses

$\downarrow$   
 Critique rationale

answer to the question

Critique of Critique :-  $C^2(Q, R_1, R_2, C_1, C_2) \rightarrow (T^2, A^2)$

$\downarrow$   
 two critiques of original responses.



Transformer Feed-forward layers are Key-value memories :-

$$FF(x) = f(x \cdot K^T) \cdot V$$

- $d_m$  key-value pairs

$$K_i \in \mathbb{R}^d \quad K \in \mathbb{R}^{d_m \times d} \quad V \in \mathbb{R}^{d_m \times d}$$

- Each key captures a particular pattern in the input sequence, and its corresponding value vector  $V_i$  represents the distribution of tokens that follow the pattern

How to prove this? 

- we analyse keys of a trained language model's feed-forward layers
- Retrieve training examples most associated with a given key

## How to retrieve trigger examples :-

$K_i^l \rightarrow$   $i$ th hidden dimension

$l^{\text{th}}$  feed-forward layer

$\rightarrow \text{ReLU}(x_j^l \cdot K_i^l)$  for every prefix  $x_1, \dots, x_j$   
of every sentence

Example :- "I love dogs"

three coefficients for three prefixes "I", "I love"  
and "I love dogs"

top- $t$  trigger examples whose representation at layer  
yielded the highest inner product with  $K_i^l$

pattern analysis :-

we let human experts annotate the trigger examples

$\rightarrow$  patterns :- "shallow" (recalling n-grams)  
"semantic" (recalling topic)

- lower layers  $\Rightarrow$  shallow patterns
- higher layers  $\Rightarrow$  semantic patterns

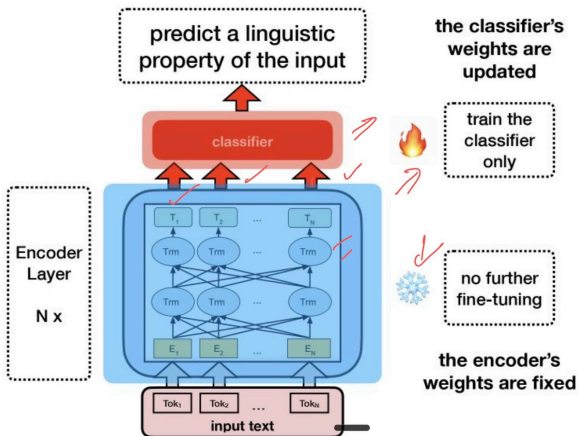
$$p_i^L = \text{softmax}(v_i^L \cdot E)$$

$\hookrightarrow$  vocabulary

$\rightarrow$  agreement rate with max-prob token and prefix with highest match increases after layer 1)

Probing  $\therefore$

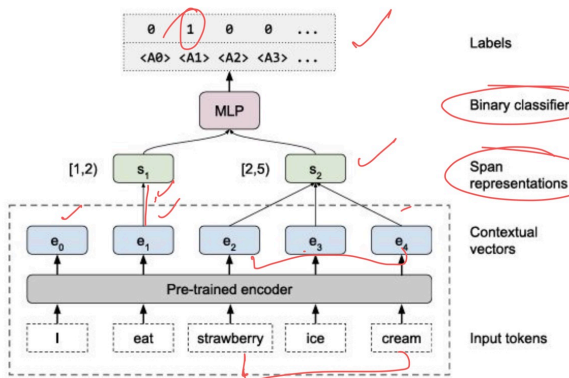
## Probe Approach



<https://www.cs.umd.edu/~mihir/cs685>

if we cannot train a classifier to predict a property of the input text based on its representation, it means property is not encoded in the representation (or) not encoded in a useful way!

## Edge Probing



Labels

Binary classifiers

Span representations

Contextual vectors

Input tokens

All parameters inside the dashed line are fixed, while we train the span pooling and MLP classifiers to extract information from the contextual vectors. The example shown is for semantic role labeling, where  $s(1) = [12]$  corresponds to the predicate ("eat"), while  $s(2) = [25]$  is the argument ("strawberry ice cream"), and we predict label A1 as positive and others as negative. For entity and constituent labeling, only a single span is used.

<https://arxiv.org/pdf/1905.06316>

$$h_{i,T} = \gamma_T \sum_{l=0}^L s_T^{(l)} h_i^{(l)}$$

↪ learnable weights

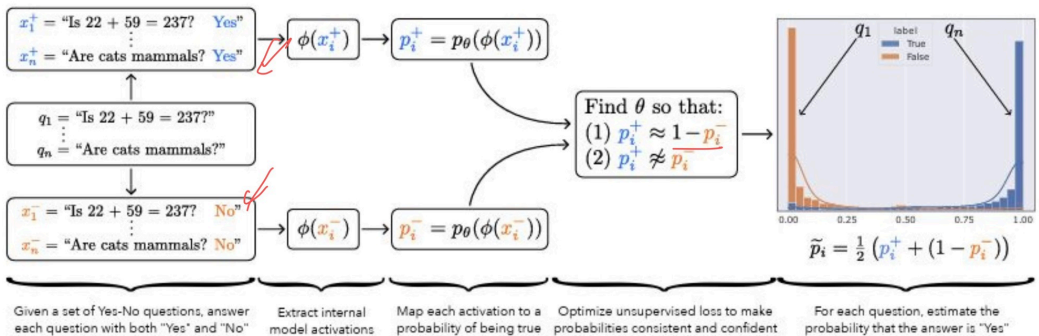
- we learn these weights jointly with probing classifier in order to allow it to extract information from many layers of an encoder

$\overline{E}_S(U) = \sum_{l=0}^L \log S_T^{(l)} \rightarrow$  A higher center of gravity means the information needed for task is captured by higher layers

## Extracting latent knowledge from language models :°

### Gain insights into the latent knowledge of the model

<https://arxiv.org/pdf/2212.03827>



$$L_{\text{consistency}}(\theta, b; q_i) := [p_{\theta, b}(x_i^+) - (1 - p_{\theta, b}(x_i^-))]^2 \quad L_{\text{confidence}}(\theta, b; q_i) := \min\{\overline{p_{\theta, b}(x_i^+)}, \overline{p_{\theta, b}(x_i^-)}\}^2$$

$$L_{\text{CCS}}(\theta, b) := \frac{1}{n} \sum_{i=1}^n L_{\text{consistency}}(\theta, b; q_i) + L_{\text{confidence}}(\theta, b; q_i)$$

# Inner Interpretability °°

Let  $(x, y)$  be the standard data matrix and output. respectively.

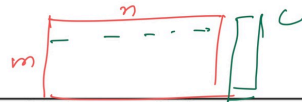
$$E(y | x=x) = x^T \beta + \beta_0$$

optimization problem :-

$$\min_{\beta} \frac{1}{2N} \|x^T \beta + \beta_0 - y\|_2^2 + \lambda R_{\alpha}(\beta)$$

$$R_{\alpha}(\beta) = (1-\alpha) \frac{1}{2} \|\beta\|_2^2 + \alpha \|\beta\|_1$$

## Explaining LMs



**Algorithm 2** Word cloud feature visualization for language models for a vocabulary  $V$ , a corpus  $w_{ij}$  for  $i, j \in [m] \times [n]$  of  $m$  sentences with  $n$  words

- 1: **for**  $i = 1 \dots m$  **do**
- 2:    $\beta_i = \text{LIME}(w_i)$  // generate LIME explanation for each sentence
- 3: **end for**
- 4: **for**  $w \in V$  **do**
- 5:    $K_w = \sum_{ij: w=w_{ij}} 1$  // count number of occurrences of word
- 6:    $\beta_w = \frac{1}{K_w} \sum_{ij: w=w_{ij}} \beta_{ij}$  // calculate average LIME explanation of word
- 7: **end for**
- 8: **return** Wordcloud( $\beta, V$ ) // generate word cloud for vocabulary  $V$  weighted by  $\beta$

## Logit Lens :-

we have a dictionary  $W$ , that lets us convert between vocab and embedding space at any point

- ① looking at the ranks
  - ② looking at KL divergence
- 

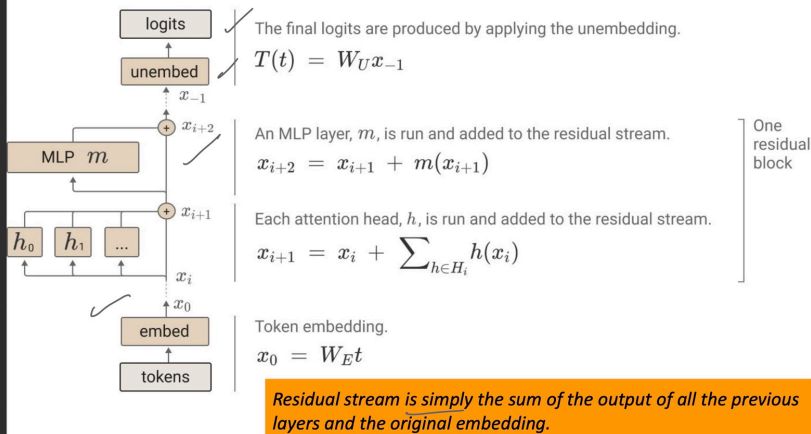
## A mathematical framework for transformer circuits :-

### Induction head hypothesis :-

- prefix matching :- The head attends back to previous tokens that were followed by (current/next) recent tokens
- copying :- head's output increases the logit corresponding to the attended-to-token

if  $[A][B] \rightarrow [A]$  occurs,  $[B] \rightarrow$  likelihood increases

# Transformer Architecture

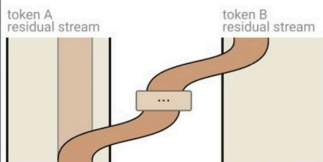


<https://transformer-circuits.pub/2021/framework/index.html>

## The residual stream

Attention heads move information across residual streams.

MLPs move information across dimensions within a residual stream.



✓ Attention heads copy information from the residual stream of one token to the residual stream of another. They typically write to a different subspace than they read from.

[https://krmopuri.github.io/xml/static\\_files/presentations/ML-Pranav.pdf](https://krmopuri.github.io/xml/static_files/presentations/ML-Pranav.pdf)

- Different heads in each layer can be thought of as operating independently of each other, reading writing into the residual stream

- heads composed of circuits

- $w_{qk} = w_q w_k^T$  called qk circuit

$\rightarrow v_i^T w_{qk} v_j \Rightarrow$  tells which tokens information is moved to and from in the residual stream

- $w_{ov} = w_v w_o$  called the ov circuit

$\rightarrow v_j^T w_{ov} \Rightarrow$  tells what information is moved from a token, if that token is attended to

in-context learning and induction heads :-

Argument-1 :- Transformers undergo a "phase change" early in training, during which induction heads form

② shifting induction heads  $\rightarrow$  shifts in-context learning

③ removing them reduce in-context learning drastically

④ they can implement more sophisticated types of in-context learning, including highly abstract behaviours

---

Alignment paradox :-

① Model finetuning :-

$$\begin{aligned} V^+(x) &\approx V(x) + C_{\text{putin}} && \text{pro-putin} \\ V^-(x) &\approx V(x) - C_{\text{putin}} && \text{anti-putin} \\ &&& \downarrow \\ &&& \text{steering vector} \end{aligned}$$

② Input finetuning

③ output finetuning

